

Lab 3: Architecture Justification

Problem Statement #42: Internal Microservice Catalog & Health Portal

Architecture Selection

We chose the **Microservices Architecture** for the Internal Microservice Catalog & Health Portal.

Architectural Choice

The Internal Microservice Catalog & Health Portal is structured as a Microservices Architecture, in which authentication, service catalog management, dependency visualization, API documentation aggregation, health monitoring, and downtime notification are separated into independent components rather than a single monolithic backend. This mirrors the portal's own domain: it manages and observes independently deployed microservices, so the portal's internal architecture follows the same independently-scalable model.

Reason 1 — Independent Scalability

Per NFR-001, the dependency graph must render interactive node architectures for up to 200 services smoothly, while the Health Monitoring Service must simultaneously issue probes every 30 seconds per FR-001. These two workloads have very different resource profiles — graph rendering is bursty and read-heavy, while health probing is a constant, high-frequency background load. Separating the Dependency Graph Service from the Health Monitoring Service allows each to be scaled independently based on its own load, rather than provisioning the entire system for the peak demand of either.

Reason 2 — Fault Isolation

Per UC-01, the Health Monitoring Service continuously probes external, unreliable Monitored Microservices, some of which will legitimately time out or fail (as seen in the Lab 2 defect where connection-refused errors were mishandled). Isolating health monitoring as its own component means a slow or failing external microservice cannot degrade the portal's own availability — the Service Catalog, Dependency Graph, and API Documentation services remain usable even while individual health probes are failing. After three consecutive failed probes, the Health Monitoring Service simply invokes the Alert & Notification Service (FR-004) to dispatch a downtime alert, without any other part of the portal being affected.

Security Advantage

Authentication and authorization are enforced by a dedicated Authentication & Authorization Service that sits in front of every other backend component. This directly satisfies NFR-002, which requires that health, dependency, and API documentation information be restricted to authenticated users: because access control is centralized in one component rather than duplicated across each service, every request for protected microservice data is verified through a single, consistently-enforced authentication boundary.

Performance Benefit

Because the Dependency Graph Service is a separate component from the Health Monitoring and Catalog services, its graph-rendering logic and data access patterns can be optimized independently — for example, through caching or pagination strategies tuned specifically for interactive graphs of up to 200 nodes (NFR-001) — without those optimizations being constrained by, or interfering with, the unrelated performance needs of the 30-second health-probing cycle.